ISIMM

# Programmation Python 1

CPI1

2026

# Plan


01 Introduction

02 Classe et objet

03 Héritage / Dérivation

04 Polymorphisme et surcharge

## 01 Introduction

En programmation procédurale, on s'intéresse aux **étapes nécessaires** pour résoudre un problème donné. Ces étapes sont modélisées par des tâches plus ou moins complexes, appelées **procédures/fonctions** qui manipulent les données du problème sans beaucoup de contrôle pour aboutir au résultat final.

→ Le programme est donc un **enchainement de ces tâches**.

## 01 Introduction

En programmation OO, on modélise le problème par un **ensemble d'objets** du monde réel qui **interagissent entre eux** à travers des **messages**. L'objet est donc l'entité principale. L'objet encapsule ses **propres données** et les **traitements relatifs** à ses données dans une même entité.

→ Le programme est **un ensemble d'objets et des interactions entre ces objets.**

## 02 Classe et objet

### A) Définitions

- Concrètement un objet est une entité du monde réel: une personne, une voiture, une forme géométrique, une image, une feuille, etc.
- En python, toutes les entités manipulées sont des objets (entier, fichier, module, etc).

## A) Définitions

La définition d'un objet (structure de l'objet) consiste à décrire ses propriétés:

- Statiques, appelées attributs ;
- Comportementales ou dynamiques, appelées méthodes

**Exemple:** Une feuille de papier possède une couleur, une longueur et une largeur et on peut la plier, la déchirer, la couper, etc.

## A) Définitions

- Un exemplaire d'un objet s'appelle une instance.
- Une feuille de papier de couleur rouge de longueur 21 cm et de largeur 17 cm est une instance de l'objet.
- Deux instances possèdent chacune des valeurs d'attributs différentes mais se partagent les mêmes méthodes de l'objet. Il n'y a aucune interférence entre les objets d'une même classe.

## 02 Classe et objet

### A) Définitions

- Une classe est une entité qui permet la création des objets (instances) ayant les mêmes propriétés.
- Pour pouvoir manipuler un objet (accéder à ses propriétés ou invoquer l'une de ses méthodes), on utilise l’opérateur point.
- L'interaction entre les objets se fait par invocation des méthodes.

## B) Création de classe en Python

Une classe permet de définir (déclarer) l'ensemble attributs et méthodes relatives à une catégorie d'objets.

- **Attributs de classe :** Un attribut de classe est défini au niveau de la classe et sa valeur est partagée par tous les objets instanciés de cette classe. L'accès à l'attribut est donné par : nom_Classe.nom_attribut
- **Attributs d'objets:** Un attribut d'objet est défini au niveau de la méthode constructeur. La valeur d'un attribut d'objet est propre à chaque objet. L'accès à l'attribut est donné par : nom_Objet.nom_attribut

## B) Création de classe en Python

- Le **constructeur d'objet:** Le constructeur est une méthode particulière appelée lors de la création d'un nouvel objet permettant d'initialiser ses attributs. Le constructeur se définit dans une classe comme une fonction avec deux particularités :

  - le nom de la méthode doit être __init__ ;
  - la méthode doit accepter au moins un paramètre de nom self qui apparaît en premier.

## B) Création de classe en Python

- Le paramètre **self:** Le paramètre self représente en fait l'objet cible, c'est-à-dire que c'est une variable qui référence l'objet en cours de création et permettant d’accéder à ses attributs et fonctionnalités.
- Les **méthodes:** Une méthode est une fonction qui agit principalement sur une instance de la classe. Elle doit accepter au moins le paramètre self figurant en première position. L’appel d’une méthode se fait par :

nom_Objet.nom_méthode (autres paramètres)

## B) Création de classe en Python

```
class nom_classe:
      #définition des attributs de classe
                  (attribut de valeur commune pour toutes les instances)
      nom_attr = valeur

      #définition des attributs d'objet (chaque instance a sa propre valeur
      def __init__(self, liste_paramètres): # le constructeur de l'objet
            self.nomattr1= v1
            self.nomattr2= v2
            …

      #définition des méthodes
      def  nom_meth (self, autres_paramètres):
            #corps de la méthode
```

## B) Création de classe en Python

**Exemple 1 :** Soit la classe Compte_bancaire définie par:

- Les attributs:
  - Attributs de classe : nombank
  - Attributs d'objet : NumeroCompte, NomProprietaire, solde
- Les méthodes: AfficherSolde, Retrait, Dépôt

## B) Création de classe en Python

```
# création de la classe Compte_Bancaire CompteBc
class compteBc:
        nombank= 'amenbank '
        def __init__(self, N, A, S)      #le paramètre self désigne l'objet lui-même
             self.num=N
             self.nom=A
             self.sold=S
        def retrait(self,x):
             if self.sold>=x:
                 self.sold= self.sold-x
             else:
                  raise Exception('retrait impossible')
        def depot(self,x):
             self.sold+=x
        def AfficherSolde (self):
             print('compte au nom de ', self.nom, 'a pour solde',self.sold)
```

## C) Création d'objets en Python

- La création d’une instance (objet) d’une classe donnée se fait par un appel au nom de la classe avec les paramètres effectifs du constructeur, selon la syntaxe suivante :

**>>> nom_obj=nom_classe(paramètres effectifs du constructeur )**

## C) Création d'objets en Python

**Exemple 2:** Soit compte1 une instance de la classe Compte_bancaire dont les données sont les suivantes :

- NumeroCompte : 512017
- NomProprietaire : mohamed
- solde : 1500

L'objet compte1 est créé ainsi :

```
>>> compte1= compteBc(512017,  mohamed, 1500)
>>> compte1.depot(500) # appel de la méthode depot
>>> compte1.AfficherSolde()
compte au nom de mohamed a pour solde 2000
```

## 03 HERITAGE / DERIVATION

### A) Définition

C’est le mécanisme qui permet de construire une classe « fille », ou sous-classe, à partir d’une classe « mère ». La classe fille ainsi obtenue hérite toutes les propriétés et toutes les fonctionnalités de son ancêtre. La classe « fille » peut éventuellement ajouter des méthodes/attributs propres à elle et redéfinir les méthodes qui ont été héritées de sa classe mère. La classe fille est une spécialisation de la classe mère. L’héritage se fait ainsi selon la syntaxe suivante:

```
class nom_sous_classe(nom_classe_mère):
            #définir les attributs et les méthodes de la sous_classe
```

## B) Exemple 3

```
class Point:
   def __init__(self,a,b):
         self.x=a
         self.y=b
   def Affiche(self):
         print('Point',(self.x, self.y))
   def deplacer(self, dx, dy) :
        self.x += dx
        self.y += dy
class PointColoré(Point):
   def __init__(self,a,b,col):
        Point.__init__(self,a,b)  # ou bien super().__init__(a,b)
        self.couleur = col
   def Affiche(self):
        print('PointColoré',(self.x, self.y,self.couleur))
```

## B) Exemple 3

```
>>> p2=PointColoré(2,3,'rouge'); p2.Affiche()
PointColoré (2, 3, 'rouge')
>>> p2.deplacer(1,1) #méthode héritée
>>> p2.Affiche()
PointColoré (3, 4, 'rouge')
```

# 04 POLYMORPHISME ET SURCHARGE

## A) Définitions

Le polymorphisme est le concept qui permet d'avoir différentes formes d'une méthode. Entre autres, il permet de redéfinir des méthodes avec des fonctionnalités similaires.

Quand ce concept est appliqué aux méthodes spéciales, on l'appelle surcharge. Une méthode spéciale est en fait une méthode dont le nom est donné par __nom__, exécutée sans appel explicite. Elle a un comportement par défaut. On peut citer comme exemples :

- __init__ : appelée lors de la création d'une instance de classe par **nom_obj=nomClasse(paramètres)**
- __str__ : appelée par print(obj)
- __add__ : appelée par obj1 + obj2

## B) Exemples

```
class Personne:
  def __init__ (self,nom):
        self.nom=nom
  def affiche(self):
        print("je suis une personne")
class Etudiant(Personne):
  def __init__(self,nom,cne):
        super().__init__(nom)
        self.cne=cne
  def affiche(self):
        print("je suis un étudiant")
```

## B) Exemples

```
>>> pr=Personne('Ali'); pr.affiche()
je suis une personne
>>> etd=Etudiant('Mohammed',123444); etd.affiche()
je suis un étudiant
```

## B) Exemples

```
class Carre :
  " " " Un carré est modélisé par son coin supérieur gauche et une arête " " "
  def  __init__ ( self, P, a ) :
        self. coin = P
        self.arete=a
  def perimetre ( self ) :
        return 4 * self.arete
  def aire ( self ) :
        return self.arete** 2

class Disque :
  " " " Un disque est modélisé par son centre et son rayon " " "
  def __init__(self, c, r ) :
        self.centre = c
        self.rayon = r
  def perimetre ( self ) :
        return 2 * pi * self.rayon
  def aire ( self ) :
        return pi * self.rayon **2
```

## B) Exemples

```
>>> from math import pi
>>> p=Point(2,3); p1=Point(0,0);d=Disque(p,5);c=Carre(p1,2)
>>> d.perimetre(); c.perimetre()
31.41592653589793
8
>>> d.aire(); c.aire()
78.53981633974483
4
```

## C) Surcharge de méthodes spéciales

- **Représentation formelle d'un objet** : Pour afficher les informations relatives à un objet, en utilisant le nom de l'objet (représentation sur le shell) ou en utilisant la commande print(), il faut surcharger la méthode spéciale __repr__ :

def __repr__(self): return #la chaine qu'on veut afficher

```
class point:
  def __init__(self,a,b):
        self.x=a
        self.y=b
  def __repr__(self):
        return (str((self.x , self.y)))
```

```
>>>p=point(2,3)
>>>p          # l'exécution de l'évaluation de p fait appel à __repr__
(2,3)
>>>print(p) # l'exécution de print fait appel à la méthode __repr__
(2,3)
```

## C) Surcharge de méthodes spéciales

- **Représentation informelle d'un objet :** Pour donner une représentation textuelle informelle à un objet, il suffit de surcharger la méthode spéciale __str__ :

def __str__(self): return #la chaine qu'on veut afficher

```
class point:
   def __init__(self,a,b):
          self.x=a
          self.y=b
   def __str__(self):
         return 'point'+str((self.x , self.y))
```

```
>>>p=point(2,3)
>>>p #sans redéfinir __repr__ l'exécution renvoie la référence de l'objet
<__main__.point object at 0x033DAB10>
>>>print(p)        # l'exécution de print fait appel à la méthode __str__
point(2,3)
```

## C) Surcharge de méthodes spéciales

```
class point:
   def __init__(self,a,b):
          self.x=a
          self.y=b
   def __repr__(self):
         return ( str((self.x , self.y)))
   def __str__(self):
         return 'point'+str((self.x , self.y))
```

```
>>>p=point(2,3)
>>>p        # l'exécution de l'évaluation de p fait appel à __repr__
(2,3)
>>>print(p) # l'exécution de print fait appel à la méthode __str__
point(2,3)
```

# Merci pour votre attention